

Notebook

February 8, 2026

1 ECDSA/DSA With a Biased Nonce

NB! This notebook should be opened with sagemath jupyter mode, not with regular python, since it uses sagemath's APIs. Install [Sagemath](#) and run:

```
sage -n jupyter
```

We've covered that DSA and ECDSA implementations are vulnerable when they are:

1. Using predictable nonces (k)
2. Using the same nonce twice

I've mentioned that using a biased nonce can also lead to private key compromise. This exercise is focused on this problem.

1.1 Intro

First, let's remind ourselves what constitutes DSA and ECDSA. For DSA we have a hash function $Hash(x)$, a multiplicative group Z_p^* modulo a prime number p and a generator g of order q , where q is the largest prime divisor of $p - 1$ (so g is generator of the largest subgroup of prime order of group Z_p^*). Private key is $x \in (0, q)$ and $y = g^x \bmod p$ is the public key.

Signing m :

1. Pick a random nonce (number used once) $k \in (0, q)$
2. Compute $r = (g^k \bmod p) \bmod q$. If $r = 0$ go to 1.
3. Compute $s = \frac{Hash(m) + x \cdot r}{k} \bmod q$. If $s = 0$ go to 1.
4. Output (r, s) . This is the signature

To verify the signature we must check, that $r, s \in (0, q)$ and the following equality:

$$(g^{\frac{Hash(m)}{s}} \cdot y^{\frac{r}{s}} \bmod p) \bmod q = r$$

In case of ECDSA we have a curve $E(GF(p))$ and some point G that belongs to the curve as generator, which has prime order q , so $q \cdot G = 0$ (Point at Infinity). We also need a hash function $Hash(x)$. To create keys we once again: 1. Pick a random $d \in (0, q)$ as our private key 2. Compute $Q = d \cdot G$ to be our public key

Signing m :

1. Pick a random nonce $k \in (0, q)$

2. Calculate $(x_1, y_1) = k \cdot G$
3. Calculate $r = x_1 \bmod q$. If $r = 0$ go to 1.
4. Calculate $s = \frac{\text{Hash}(m) + r \cdot d}{k} \bmod q$. If $s = 0$ go to 1.
5. Output (r, s) as signature.

Verification:

1. Check that $r, s \in (0, q)$
2. Compute $(x_1, y_1) = \frac{\text{Hash}(m)}{s} \cdot G + \frac{r}{s} \cdot Q$
3. Check that $r = x_1 \bmod q$

I skipped correctness this time, since we've already covered it last time and it won't help you with the attack, while looking at the signing and verification procedures is important.

So what do we mean, when we talk about a *biased* nonce? We imply that the distribution of nonces is biased toward a certain region. For example, the top 8 bits of k are always zero. We know that $s = \frac{\text{Hash}(m) + x \cdot r}{k} \bmod q$, then

$$s \cdot k = \text{Hash}(m) + x \cdot r \bmod q$$

$$k = \frac{\text{Hash}(m) + x \cdot r}{s} \bmod q$$

$$x \cdot \frac{-r}{s} = \frac{\text{Hash}(m)}{s} - k \bmod q$$

We can rewrite this equation into the form

$x \cdot a = b - c \bmod q$, where $a = \frac{-r}{s} \bmod q$, $b = \frac{\text{Hash}(m)}{s}$ and $c = k$. Notice that c is much smaller than x , a and b , so we can say that $x \cdot a \simeq b \bmod q$ (almost equal).

1.2 HNP, CVP, LLL and Gram-Schmidt

What we are looking at is a variant of the Hidden Number Problem (HNP):

Recover $\alpha \in F_p$ such that for many known random $t \in F_p$ we are given $MSB_{l,p}(\alpha t)$ for some $l > 0$. The algorithm for solving this is based on closest vector problem. To formulate that we first need to know, what lattices are.

Let $\{b_1, \dots, b_s\}$ be a set of linearly independent vectors in $\mathbb{R}^s$.

The set of vectors

$$L = \{z | z = \sum_{i=1}^s c_i b_i, c_1, \dots, c_s \in \mathbb{Z}\}$$

is called an s -dimensional full rank lattice. The set $\{b_1, \dots, b_s\}$ is called a basis of L .

The Closest Vector Problem (CVP):

Given a vector $r \in \mathbb{R}^s$ find a lattice vector $v \in L$ with

$$\|r - v\| = \min_{z \in L} \|r - z\|$$

CVP is NP-complete, but there are polynomial-time algorithms that can find an approximate solution, such as LLL (Lenstra, Lenstra, Lovasz).

LLL uses Gram-Schmidt process. It is a method for orthonormalizing a set of vectors in an inner product space (scalar multiplication is defined).

$u, v \in \mathbb{R}^s$, a projection operator is defined by $proj_u(v) = \frac{(u,v)}{(u,u)}u$, where (u,v) denotes the inner product (scalar multiplication). Then for a given set of vectors $v_1, \dots, v_k \in \mathbb{R}^s$ Gram-Schmidt first performs orthogonalization:

$$\begin{aligned} u_1 &= v_1 \\ u_2 &= v_2 - proj_{u_1}(v_2) \\ &\dots \\ u_k &= v_k - \sum_{j=1}^{k-1} proj_{u_j}(v_k) \end{aligned}$$

The goal is to make sure that the resulting basis is orthogonal (the scalar multiplication of each pair of distinct vectors is equal to zero). Then each vector is divided by the square root of its scalar multiplication by itself, to normalize them and get the final orthonormalized basis.

The LLL algorithm computes LLL-reduced bases. Given a basis

$$B = \{b_1, b_2, \dots, b_n\}$$

its Gram-Schmidt process orthogonal (no normalization) basis

$$B^* = \{b_1^*, b_2^*, \dots, b_n^*\}$$

and the Gram-Schmidt coefficients

$$\mu_{i,j} = \frac{(b_i, b_j^*)}{(b_j^*, b_j^*)}, \text{ for any } 1 \leq j < i \leq n.$$

Then the basis B is LLL-reduced if there exists a parameter $\delta \in (0.25, 1]$ such that the following holds:

1. (size-reduced) For $1 \leq j < i \leq n : |\mu_{i,j}| \leq 0.5$. This property guarantees the length reduction of the ordered basis.
2. (Lovasz condition) For $k = 2, 3, \dots, n : \delta ||b_{k-1}^*||^2 \leq ||b_k^*||^2 + \mu_{k,k-1}^2 ||b_{k-1}^*||^2$.

Basically, δ shows how well-reduced the basis is. Here is LLL pseudocode in python-like syntax. B is a matrix with each row representing a separate basis vector.

```
def LLL(B, delta):
    B = copy(B)
    Q = gram_schmidt(B)
    def mu(i, j):
        v = B[i]
        u = Q[j]
        return (v*u) / (u*u)
```

```

n = len(B)
k = 1

while k < n:
    for j in reverse(range(k)):
        if abs(mu(k, j)) > 1/2:
            B[k] := B[k] - round(mu(k, j))*B[j]
            Q := gramshmidt(B)

    if (Q[k]*Q[k]) >= (delta - mu(k, k-1)^2) * (Q[k-1]*Q[k-1]):
        k := k + 1
    else:
        B[k], B[k-1] := B[k-1], B[k]
        Q := gramshmidt(B)
        k := max(k-1, 1)

return B

```

You can read more about LLL and take a peek at the algorithm internals on [wiki](#). The basic idea is that it attempts to reduce the basis by reordering and subtracting vectors from each other. As a result each vector in the new basis will be a linear combination of vectors in the initial basis.

1.3 Solving DSA/ECDSA biased nonces with LLL

Remember that $x \cdot a \simeq b \pmod q$, which means that $x \cdot a \simeq b + j \cdot q, j \in \mathbb{Z}$. What we need is to find x . a, b and q are known. So for several $a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_n$ we can build the following matrix, which will represent a set of vectors:

$$\begin{vmatrix} q & 0 & 0 & \dots & 0 \\ 0 & q & 0 & \dots & 0 \\ 0 & 0 & q & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & q \\ a_1 & a_2 & a_3 & \dots & a_n \\ b_1 & b_2 & b_3 & \dots & b_n \end{vmatrix}$$

Each row is a vector. We know that $x \cdot \vec{a} \simeq \vec{b}$ (here coefficients are in F_q), so one of the basis vectors that LLL will compute will be $\vec{c} = \vec{b} - x \cdot \vec{a} + \sum_{i=1}^n (t_i \vec{q}_i), t_i \in \mathbb{Z}$ (here the coefficients are in $\mathbb{Q}$). But what we actually want to compute is x , so how do we do that? Simple, extend the vectors by two elements. For the vectors containing q set those elements to zero. We want to balance the final vector, so that all the elements would be roughly the same. If top l bits of k are biased, then for $\vec{a}$ we can set the $(n+1)$ -th element to $\frac{1}{2^l} \in \mathbb{Q}$ and for $\vec{b}$ we can set the $(n+2)$ -nd element to $\frac{q}{2^l}$. Since the final vector should be $\vec{b} - x \cdot \vec{a}$ the last element will be $\frac{q}{2^l}$ and the second to last will be

$\frac{-x}{2^l}$ so all the elements will have roughly the same rank.

$$\begin{vmatrix} q & 0 & 0 & \dots & 0 & 0 & 0 \\ 0 & q & 0 & \dots & 0 & 0 & 0 \\ 0 & 0 & q & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & q & 0 & 0 \\ a_1 & a_2 & a_3 & \dots & a_n & \frac{1}{2^l} & 0 \\ b_1 & b_2 & b_3 & \dots & b_n & 0 & \frac{q}{2^l} \end{vmatrix}$$

This way in the end we can check all rows (basis vectors) for the last element equal to $\frac{q}{2^l}$ and if one of them has it, then the second element from the end will with high probability contain the value $\frac{-x}{2^l}$ from which we can easily compute the x .

As you can see, there are four steps in this attack:

1. Collect enough (r, s) pairs (their number depends on the bias)
2. Compute vectors $\vec{a}$ and $\vec{b}$ and construct the lattice (matrix)
3. Use LLL for solving SVP (finding the vector closest to $\vec{0}$)
4. Compute x from resulting basis

Using LLL is easy:

```
[ ]: from sage.all import *
# You will need rational numbers:
print(Rational(1)/2)
M=matrix([[0,1,1],[2,3,1],[1,1,1]])
print(M.LLL())
```

1.4 Back to Biases

What about the case when low bits of k are biased? For example, lowest l bits of k are set to a value z :

$$k = 2^l \cdot k_0 + z.$$

$$s = \frac{\text{Hash}(m) + x \cdot r}{k} \bmod q$$

$$k = \frac{\text{Hash}(m) + x \cdot r}{s} \bmod q$$

$$2^l \cdot k_0 + z = \frac{\text{Hash}(m) + x \cdot r}{s} \bmod q$$

$$2^l \cdot k_0 = \left(\frac{\text{Hash}(m)}{s} - z \right) + \frac{x \cdot r}{s} \bmod q$$

$$k_0 = \frac{(\frac{Hash(m)}{s} - z)}{2^l} + \frac{x \cdot r}{s \cdot 2^l} \bmod q$$

$$x \cdot \frac{-r}{s \cdot 2^l} = \frac{(\frac{Hash(m)}{s} - z)}{2^l} - k_0 \bmod q$$

We once again have $a \cdot x = b - c$ where $c = k_0$ is much smaller than both $a = \frac{-r}{s \cdot 2^l}$ and $b = \frac{(\frac{Hash(m)}{s} - z)}{2^l}$. So we can use the same method.

1.5 Task

The server generates two keys. It uses the first key to sign 80 messages with MSB-biased variant and the second one to sign 80 messages with LSB-biased variant. There should be more than enough signatures to recover the keys (you can look up biases in the code below). Your goal is to recover the keys and sign the messages 'give' and 'flag' like in the previous task. I recommend that you first write a function for generic $\vec{a}$ and $\vec{b}$ and then write transformations from $Hash(m), r, s$ into a, b . That way you can reuse your code. Also, don't forget that you can implement your own server for testing.

Good luck!

```
[ ]: import random
import time
from Crypto.Util.number import bytes_to_long
import hashlib
#These are the classes used on the server's side
class DSA:
    def __init__(self,g,p,q):
        self.p=p
        self.g=g
        self.q=q
        self.x=random.randint(1,q-1)
        self.y=pow(g,self.x,self.p)

    def get_k(self):
        raise NotImplementedError()
    @staticmethod
    def message_to_hashnum(m):
        if not isinstance(m,bytes):
            raise TypeError()
        return bytes_to_long(hashlib.sha256(m).digest())
    def sign(self,m):
        r=0
        s=0
        while r==0 or s==0:
            k=0
```

```

        while k==0:
            k=self.get_k()
            r=pow(g,k,self.p)%self.q
            s=((DSA.message_to_hashnum(m)+(self.x*r)%self.q)%self.
↪ q)*pow(k,q-2,self.q))%self.q
            return (r,s)
        def verify(self,m,r,s,otherY):
            if r==0 or s==0: return False
            w=pow(s,self.q-2,self.q)
            u1=(DSA.message_to_hashnum(m)*w)%self.q
            u2=(r*w)%self.q
            res=(pow(self.g,u1,self.p)*pow(otherY,u2,self.p)%p)%q
            return res==r

class MSBBIasedDSA(DSA):
    def get_k(self):
        return random.randint(0,self.q>>32)

class LSBBIased_DSA(DSA):
    def get_k(self):
        return random.randint(0,self.q-1)|((1<<32)-1)

```

```

[ ]: import socket
import re
class VulnServerClient:
    def __init__(self,show=True):
        """Initialization, connecting to server"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1351))

    def recv_until(self,symb=b'\n>'):
        """Receive messages from server, by default till new prompt"""
        data=b''
        while True:
            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def getChallenge(self,show=True):
        print ("Please wait. Signature generation can take some time..
↪.")
        data=self.recv_until()
        try:
            data=data.decode()
        except UnicodeDecodeError:

```

```

        print ('Error decoding unicode. Try connecting to server_
↪again.')
```

```

        return (None, None)
    if show:
        print (data)
        p=int(re.search(r'(?<=p=)\d+', data).group(0))
        q=(p-1)//2
        g=int(re.search(r'(?<=g=)\d+', data).group(0))
        y1=int(re.search(r'(?<=y1=)\d+', data).group(0))
        y2=int(re.search(r'(?<=y2=)\d+', data).group(0))
        all_tuples=re.findall(r'(?<=(m,r,s\=))\([a-
z]+\, \d+, \d+)\)', data)
        top_biased=[]
        for i in range(0,80):
            spl=all_tuples[i][1:-1].split(',')
            top_biased.append((spl[0].
↪encode(),int(spl[1]),int(spl[2])))

        low_biased=[]
        for i in range(80,160):
            spl=all_tuples[i][1:-1].split(',')
            low_biased.append((spl[0].
↪encode(),int(spl[1]),int(spl[2])))
        print ('Received challenge')
        return (p,q,g,y1,y2,top_biased,low_biased)

    def checkSolution(self,rg,sg,rf,sf, show=True):
        self.s.sendall((str(rg)+' '+str(sg)+' '+str(rf)+'_
↪'+str(sf)+'\n').encode())
        data=self.recv_until(b'\n')
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Error decoding unicode. Try connecting to server_
↪again.')
```

```

        return None
    if show:
        print (data)
    if data.find('flag')!=-1:
        return True
    else:
        data=self.recv_until(b'>')
        try:
            data=data.decode()
        except UnicodeDecodeError:

```



```

        print ('Error decoding unicode. Try connecting to_
server again.')
        return None
    if show:
        print (data)
        return False
def __del__(self):
    self.s.close()

vs=VulnServerClient()
(p,q,g,y1,y2,msb_biased,lsb_biased)=vs.getChallenge(show=False)

```

```

[ ]: bias= # look in the code
a_vector=[]
b_vector=[]

# Transform (H(m),r,s)-> (a,b) for MSB case
# H = DSA.message_to_hashnum

def SolveForAB(a_vector,b_vector,bias,g,y,p,q):
    # Create the Matrix, use LLL, search in vectors
    return x
x1=SolveForAB(a_vector,b_vector,bias,g,y1,p,q)

```

```

[ ]: a_vector=[]
b_vector=[]

# Transform (H(m),r,s)-> (a,b) for LSB case

x2=SolveForAB(a_vector,b_vector,bias,g,y2,p,q)

```

```

[ ]: lDSA=LSBBIased_DSA(g,p,q)
lDSA.x=x1
lDSA.y=y1
(r_g,s_g)=lDSA.sign(b'give')
lDSA.x=x2
lDSA.y=y2
(r_f,s_f)=lDSA.sign(b'flag')

```

```

[ ]: vs.checkSolution(r_g,s_g,r_f,s_f)

```

```

[ ]:

```